# 03 — Modelado de riesgo de mora

## Give Me Some Credit — Análisis de riesgo crediticio

**Autor:** Mario Zarza Zárraga
**Entrada:** `data/cs-training-limpio.csv`
**Objetivo:** comparar un baseline ingenuo, una **regresión logística** y un **modelo de árboles** (HistGradientBoosting) para predecir `mora_grave`, elegir un umbral con criterio de negocio y dejar recomendaciones claras para un gerente fintech.

Todo está pensado para poder explicarlo en una entrevista junior: pocas features, justificación simple y métricas que importan cuando hay desbalance.

## 0. Librerías y datos

In [1]:
```
import warnings
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.calibration import CalibratedClassifierCV
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
    average_precision_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_recall_curve,
    precision_score,
    recall_score,
    roc_auc_score,
    roc_curve,
)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore", category=FutureWarning)
```

```
ROOT = Path("..")
DATA = ROOT / "data"
FIG = ROOT / "reports" / "figures"
FIG.mkdir(parents=True, exist_ok=True)
REPORTS = ROOT / "reports"

sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams.update({
    "figure.figsize": (9, 5),
    "axes.titlesize": 13,
    "axes.labelsize": 11,
    "axes.titleweight": "bold",
    "font.size": 10,
})

RANDOM_STATE = 42
COLOR_MORA = "#c0392b"
COLOR_OK = "#2980b9"
```

In [2]:
```
df = pd.read_csv(DATA / "cs-training-limpio.csv")

# Feature combinada (igual que en el EDA)
df["total_atrasos"] = (
    df["atrasos_30_59"] + df["atrasos_60_89"] + df["atrasos_90"]
).astype(int)

print(f"Forma: {df.shape[0]:,} × {df.shape[1]}")
print(f"Tasa de mora: {df['mora_grave'].mean():.2%}")
df.head()
```

```
Forma: 149,999 × 16
Tasa de mora: 6.68%
```

Out[2]:

| | mora_grave | uso_credito_revolvente | edad | atrasos_30_59 | ratio_deuda | ingreso_men |
|---|---|---|---|---|---|---|
| **0** | 1 | 0.766127 | 45 | 2.0 | 0.802982 | 91 |
| **1** | 0 | 0.957151 | 40 | 0.0 | 0.121876 | 26 |
| **2** | 0 | 0.658180 | 38 | 1.0 | 0.085113 | 30 |
| **3** | 0 | 0.233810 | 30 | 0.0 | 0.036050 | 33 |
| **4** | 0 | 0.907239 | 49 | 1.0 | 0.024926 | 635 |

# 1. Features y justificación

### Qué entramos al modelo

| Feature | Por qué |
|---|---|
| `total_atrasos` | #1 en IV del EDA; resume el historial sin meter las 3 columnas a la vez |

| Feature | Por qué |
|---|---|
| `codigo_atraso_especial` | Bandera 96/98: pocos casos, pero mora ~54% |
| `uso_credito_revolvente` | #2 en IV / #1 en AUC univariado |
| `uso_revolvente_gt1` | Marca usos anómalos > 100% |
| `edad` | Señal media y fácil de explicar |
| `ingreso_mensual` + `ingreso_faltante` | Ingreso imputado + bandera de que faltaba |
| `ratio_deuda` + `ratio_deuda_dudoso` | Endeudamiento; bandera cuando el ratio no es comparable |
| `lineas_credito_abiertas`, `prestamos_hipotecarios`, `dependientes` | Contexto de exposición / perfil |

**Qué NO entramos (y por qué):** las tres columnas crudas `atrasos_30_59`, `atrasos_60_89`, `atrasos_90`. En el EDA se correlacionan entre sí (multicolinealidad). En una logística eso infla e inestabiliza coeficientes. Usamos `total_atrasos` + la bandera especial.

In [3]:
```
FEATURES = [
    "total_atrasos",
    "codigo_atraso_especial",
    "uso_credito_revolvente",
    "uso_revolvente_gt1",
    "edad",
    "ingreso_mensual",
    "ingreso_faltante",
    "ratio_deuda",
    "ratio_deuda_dudoso",
    "lineas_credito_abiertas",
    "prestamos_hipotecarios",
    "dependientes",
]
TARGET = "mora_grave"

X = df[FEATURES].copy()
y = df[TARGET].copy()

print("Features:", FEATURES)
print(f"X: {X.shape}, y positiva: {y.mean():.2%}")
```

```
Features: ['total_atrasos', 'codigo_atraso_especial', 'uso_credito_revolvent
e', 'uso_revolvente_gt1', 'edad', 'ingreso_mensual', 'ingreso_faltante', 'ra
tio_deuda', 'ratio_deuda_dudoso', 'lineas_credito_abiertas', 'prestamos_hipo
tecarios', 'dependientes']
X: (149999, 12), y positiva: 6.68%
```

# 2. Train / test estratificado (80/20)

Usamos `stratify=y` para que train y test tengan **la misma proporción de mora** (~6.7%).
`random_state=42` para que el resultado sea reproducible.

In [4]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y
)

print(f"Train: {len(X_train):,} | mora={y_train.mean():.2%}")
print(f"Test:  {len(X_test):,} | mora={y_test.mean():.2%}")
```

```
Train: 119,999 | mora=6.68%
Test:  30,000 | mora=6.68%
```

# 3. Baseline: «nadie cae en mora»

Si predecimos siempre 0 (sin mora), la **accuracy** se ve alta (~93%)... pero el recall de mora es 0.
Eso es la **trampa de la accuracy** con clases desbalanceadas: un modelo inútil puede parecer bueno.

In [5]:
```
# Baseline: siempre predice 0 (sin mora)
y_pred_base = np.zeros_like(y_test)
# Probabilidad de mora = 0 para todos (no ordena riesgo)
y_proba_base = np.zeros(len(y_test))

acc_base = accuracy_score(y_test, y_pred_base)
rec_base = recall_score(y_test, y_pred_base, zero_division=0)
prec_base = precision_score(y_test, y_pred_base, zero_division=0)
f1_base = f1_score(y_test, y_pred_base, zero_division=0)
# AUC no está definido con scores constantes; lo marcamos como N/A (0.5 por
auc_base = 0.5

print("=== Baseline (siempre 0) ===")
print(f"Accuracy:  {acc_base:.2%}  ← se ve 'bien' por el desbalance")
print(f"Precision: {prec_base:.2%}")
print(f"Recall:    {rec_base:.2%}  ← no detecta a nadie en mora")
print(f"F1:        {f1_base:.2%}")
print(f"AUC:       {auc_base:.3f} (scores constantes → no ordena)")
print("\nMatriz de confusión (filas=real, cols=predicho):")
print(confusion_matrix(y_test, y_pred_base))
```

```
=== Baseline (siempre 0) ===
Accuracy:  93.32%  ← se ve 'bien' por el desbalance
Precision: 0.00%
Recall:    0.00%  ← no detecta a nadie en mora
F1:        0.00%
AUC:       0.500 (scores constantes → no ordena)

Matriz de confusión (filas=real, cols=predicho):
[[27995     0]
 [ 2005     0]]
```

# 4. Regresión logística (escalada + `class_weight='balanced'`)

- **StandardScaler:** la logística es sensible a la escala (ingreso en miles vs. uso 0–1).
- **`class_weight='balanced'`:** da más peso a la clase minoritaria (mora) para que el modelo no ignore a los defaulters.

In [6]:
```
# Pipeline: escalar numéricas + logística balanceada
logit = Pipeline(
    steps=[
        ("scaler", StandardScaler()),
        (
            "clf",
            LogisticRegression(
                class_weight="balanced",
                max_iter=1000,
                random_state=RANDOM_STATE,
                solver="lbfgs",
            ),
        ),
    ]
)

logit.fit(X_train, y_train)
y_proba_logit = logit.predict_proba(X_test)[:, 1]
y_pred_logit = (y_proba_logit >= 0.5).astype(int)

print("Logística entrenada. Predicción por defecto con umbral 0.5.")
```

```
Logística entrenada. Predicción por defecto con umbral 0.5.
```

In [7]:
```
# Coeficientes e odds ratios (sobre features estandarizadas)
coef = logit.named_steps["clf"].coef_.ravel()
intercept = float(logit.named_steps["clf"].intercept_[0])
coef_df = pd.DataFrame({
    "feature": FEATURES,
    "coeficiente": coef,
    "odds_ratio": np.exp(coef),
}).sort_values("coeficiente", ascending=False)

print(f"Intercepto: {intercept:.4f}")
print("\nCoeficientes (feature estandarizada → +1 desvío estándar):")
print("Odds ratio > 1: sube la probabilidad de mora; < 1: la baja.")
coef_df.round(4)
```

```
Intercepto: -0.7227

Coeficientes (feature estandarizada → +1 desvío estándar):
Odds ratio > 1: sube la probabilidad de mora; < 1: la baja.
```

Out[7]:

| | feature | coeficiente | odds_ratio |
|---|---|---|---|
| **0** | total_atrasos | 0.8202 | 2.2709 |
| **2** | uso_credito_revolvente | 0.7228 | 2.0602 |
| **9** | lineas_credito_abiertas | 0.1525 | 1.1647 |
| **10** | prestamos_hipotecarios | 0.1448 | 1.1558 |
| **1** | codigo_atraso_especial | 0.1050 | 1.1107 |
| **3** | uso_revolvente_gt1 | 0.0285 | 1.0289 |
| **11** | dependientes | 0.0190 | 1.0192 |
| **6** | ingreso_faltante | 0.0159 | 1.0160 |
| **8** | ratio_deuda_dudoso | 0.0159 | 1.0160 |
| **7** | ratio_deuda | -0.1149 | 0.8915 |
| **5** | ingreso_mensual | -0.1892 | 0.8276 |
| **4** | edad | -0.2841 | 0.7527 |

In [8]:
```
fig, ax = plt.subplots(figsize=(9, 5.5))
plot_c = coef_df.sort_values("coeficiente")
colors = [COLOR_MORA if v > 0 else COLOR_OK for v in plot_c["coeficiente"]]
ax.barh(plot_c["feature"], plot_c["coeficiente"], color=colors)
ax.axvline(0, color="black", lw=0.8)
ax.set_xlabel("Coeficiente (escala estandarizada)")
ax.set_title("Regresión logística — coeficientes")
fig.tight_layout()
fig.savefig(FIG / "11_logit_coeficientes.png", dpi=140, bbox_inches="tight")
plt.show()
```

```
/tmp/ipykernel_81677/1468834892.py:10: UserWarning: FigureCanvasAgg is non-i
nteractive, and thus cannot be shown
  plt.show()
```

# 5. Modelo de árboles: HistGradientBoosting

No necesita escalado, tolera no-linealidades e interacciones, y es menos sensible a la multicolinealidad.
Usamos `class_weight` -equivalente vía `sample_weight` balanceado en el fit (HGB no tiene `class_weight` directo en todas las versiones).

In [9]:
```
# Pesos balanceados por clase (equivalente a class_weight='balanced')
n_pos = int(y_train.sum())
n_neg = int(len(y_train) - n_pos)
w_pos = len(y_train) / (2 * n_pos)
w_neg = len(y_train) / (2 * n_neg)
sample_w = np.where(y_train == 1, w_pos, w_neg)
```

```
hgb = HistGradientBoostingClassifier(
    max_depth=6,
    learning_rate=0.08,
    max_iter=200,
    random_state=RANDOM_STATE,
    early_stopping=True,
    validation_fraction=0.1,
    n_iter_no_change=15,
)
hgb.fit(X_train, y_train, sample_weight=sample_w)

y_proba_hgb = hgb.predict_proba(X_test)[:, 1]
y_pred_hgb = (y_proba_hgb >= 0.5).astype(int)
print(f"HGB entrenado. Iteraciones usadas: {hgb.n_iter_}")
print(f"Pesos: w_pos={w_pos:.3f}, w_neg={w_neg:.3f}")
```

```
HGB entrenado. Iteraciones usadas: 60
Pesos: w_pos=7.480, w_neg=0.536
```

# 6. Métricas en test (umbral 0.5)

In [10]:
```
def eval_model(name, y_true, y_pred, y_proba):
    out = {
        "modelo": name,
        "accuracy": accuracy_score(y_true, y_pred),
        "precision": precision_score(y_true, y_pred, zero_division=0),
        "recall": recall_score(y_true, y_pred, zero_division=0),
        "f1": f1_score(y_true, y_pred, zero_division=0),
        "auc": roc_auc_score(y_true, y_proba) if len(np.unique(y_proba)) > 1
        "ap": average_precision_score(y_true, y_proba) if len(np.unique(y_pr
    }
    return out


metrics = pd.DataFrame([
    eval_model("Baseline (siempre 0)", y_test, y_pred_base, y_proba_base),
    eval_model("Logística balanceada", y_test, y_pred_logit, y_proba_logit),
    eval_model("HistGradientBoosting", y_test, y_pred_hgb, y_proba_hgb),
])
metrics_round = metrics.copy()
for c in ["accuracy", "precision", "recall", "f1", "auc", "ap"]:
    metrics_round[c] = metrics_round[c].map(lambda v: f"{v:.3f}")
print("=== Métricas en TEST (umbral 0.5) ===")
print(metrics_round.to_string(index=False))
metrics.to_csv(REPORTS / "metricas_test_umbral05.csv", index=False)
```

```
=== Métricas en TEST (umbral 0.5) ===
             modelo accuracy precision recall    f1   auc    ap
Baseline (siempre 0)    0.933     0.000  0.000 0.000 0.500 0.067
Logística balanceada    0.804     0.219  0.753 0.339 0.859 0.379
HistGradientBoosting    0.793     0.214  0.786 0.336 0.868 0.391
```

In [11]:
```
# Matrices de confusión lado a lado (logit vs HGB)
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
```

```
for ax, pred, title in [
    (axes[0], y_pred_logit, "Logística (umbral 0.5)"),
    (axes[1], y_pred_hgb, "HistGradientBoosting (umbral 0.5)"),
]:
    cm = confusion_matrix(y_test, pred)
    disp = ConfusionMatrixDisplay(cm, display_labels=["Sin mora", "Mora"])
    disp.plot(ax=ax, cmap="Blues", colorbar=False)
    ax.set_title(title)
fig.tight_layout()
fig.savefig(FIG / "12_matrices_confusion.png", dpi=140, bbox_inches="tight")
plt.show()

print("Logística — classification report:")
print(classification_report(y_test, y_pred_logit, target_names=["Sin mora",
print("HGB — classification report:")
print(classification_report(y_test, y_pred_hgb, target_names=["Sin mora", "M
```

```
Logística — classification report:
              precision    recall  f1-score   support

    Sin mora       0.98      0.81      0.88     27995
        Mora       0.22      0.75      0.34      2005

    accuracy                           0.80     30000
   macro avg       0.60      0.78      0.61     30000
weighted avg       0.93      0.80      0.85     30000

HGB — classification report:
              precision    recall  f1-score   support

    Sin mora       0.98      0.79      0.88     27995
        Mora       0.21      0.79      0.34      2005

    accuracy                           0.79     30000
   macro avg       0.60      0.79      0.61     30000
weighted avg       0.93      0.79      0.84     30000
```

```
/tmp/ipykernel_81677/4027623868.py:13: UserWarning: FigureCanvasAgg is non-i
nteractive, and thus cannot be shown
  plt.show()
```

In [12]:
```
# Curvas ROC
fig, ax = plt.subplots(figsize=(7.5, 6))
for name, proba, color in [
    ("Logística", y_proba_logit, COLOR_OK),
    ("HistGradientBoosting", y_proba_hgb, COLOR_MORA),
]:
    fpr, tpr, _ = roc_curve(y_test, proba)
    auc = roc_auc_score(y_test, proba)
    ax.plot(fpr, tpr, color=color, lw=2, label=f"{name} (AUC={auc:.3f})")
ax.plot([0, 1], [0, 1], "k--", lw=1, label="Azar (AUC=0.5)")
ax.set_xlabel("False Positive Rate")
ax.set_ylabel("True Positive Rate (Recall)")
ax.set_title("Curva ROC — test")
ax.legend(loc="lower right")
fig.tight_layout()
```

```
fig.savefig(FIG / "13_curva_roc.png", dpi=140, bbox_inches="tight")
plt.show()
```

```
/tmp/ipykernel_81677/1398875313.py:17: UserWarning: FigureCanvasAgg is non-i
nteractive, and thus cannot be shown
  plt.show()
```

In [13]:
```
# Curvas Precision-Recall (más informativas con desbalance)
fig, ax = plt.subplots(figsize=(7.5, 6))
baseline_ap = y_test.mean()
for name, proba, color in [
    ("Logística", y_proba_logit, COLOR_OK),
    ("HistGradientBoosting", y_proba_hgb, COLOR_MORA),
]:
    prec, rec, _ = precision_recall_curve(y_test, proba)
    ap = average_precision_score(y_test, proba)
    ax.plot(rec, prec, color=color, lw=2, label=f"{name} (AP={ap:.3f})")
ax.axhline(baseline_ap, color="gray", ls="--", lw=1, label=f"Baseline tasa m
ax.set_xlabel("Recall")
ax.set_ylabel("Precision")
ax.set_title("Curva Precision–Recall — test")
ax.legend(loc="upper right")
fig.tight_layout()
fig.savefig(FIG / "14_curva_pr.png", dpi=140, bbox_inches="tight")
plt.show()
```

```
/tmp/ipykernel_81677/1202476344.py:18: UserWarning: FigureCanvasAgg is non-i
nteractive, and thus cannot be shown
  plt.show()
```

# 7. Umbral de decisión (no siempre 0.5)

El `predict_proba` da un **score de riesgo**. El umbral decide a quién rechazamos / revisamos.

**Ejemplo ilustrativo de costo (supuesto, no es un dato real del dataset):**

- Costo de **dejar pasar un defaulter** (falso negativo) = **5×**
- Costo de **rechazar a un buen cliente** (falso positivo) = **1×** (margen perdido)

Costo esperado ≈ `5 × FN + 1 × FP` (en la muestra de test).
Elegimos el umbral que **minimiza ese costo ilustrativo**. En la vida real, finanzas/riesgo fijan estos pesos.

In [14]:
```
# Usamos HGB (mejor AUC) para la tabla de umbrales
proba = y_proba_hgb
thresholds = [0.20, 0.30, 0.40, 0.50, 0.60, 0.70]

COST_FN = 5  # supuesto ilustrativo
COST_FP = 1

rows = []
for t in thresholds:
```

```
    pred = (proba >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, pred).ravel()
    rows.append({
        "umbral": t,
        "precision": precision_score(y_true=y_test, y_pred=pred, zero_divisi
        "recall": recall_score(y_test, pred, zero_division=0),
        "f1": f1_score(y_test, pred, zero_division=0),
        "pct_rechazados": pred.mean(),
        "TP": int(tp),
        "FP": int(fp),
        "FN": int(fn),
        "TN": int(tn),
        "costo_ilustrativo": COST_FN * fn + COST_FP * fp,
    })

thr_df = pd.DataFrame(rows)
print("=== Tabla de umbrales (modelo HGB) ===")
print("(costo_ilustrativo = 5×FN + 1×FP — SUPUESTO de ejemplo)")
print(thr_df.to_string(index=False))
thr_df.to_csv(REPORTS / "tabla_umbrales_hgb.csv", index=False)

best = thr_df.loc[thr_df["costo_ilustrativo"].idxmin()]
CHOSEN_THRESHOLD = float(best["umbral"])
print(f"\nUmbral elegido (mín. costo ilustrativo): {CHOSEN_THRESHOLD:.2f}")
print(f"  recall={best['recall']:.1%} | precision={best['precision']:.1%} |
print(f"  costo ilustrativo={int(best['costo_ilustrativo']):,}")
```

```
=== Tabla de umbrales (modelo HGB) ===
(costo_ilustrativo = 5×FN + 1×FP — SUPUESTO de ejemplo)
 umbral  precision   recall       f1  pct_rechazados   TP    FP  FN    TN  c
osto_ilustrativo
    0.2   0.117801 0.940648 0.209381        0.533667 1886 14124 119 13871
14719
    0.3   0.147080 0.900748 0.252870        0.409300 1806 10473 199 17522
11468
    0.4   0.176331 0.853865 0.292300        0.323633 1712  7997 293 19998
9462
    0.5   0.213849 0.785536 0.336179        0.245500 1575  5790 430 22205
7940
    0.6   0.273280 0.695262 0.392344        0.170033 1394  3707 611 24288
6762
    0.7   0.333707 0.594514 0.427470        0.119067 1192  2380 813 25615
6445

Umbral elegido (mín. costo ilustrativo): 0.70
  recall=59.5% | precision=33.4% | % rechazados=11.9%
  costo ilustrativo=6,445
```

In [15]:
```
fig, ax1 = plt.subplots(figsize=(8.5, 5))
ax1.plot(thr_df["umbral"], thr_df["recall"], "o-", color=COLOR_MORA, label="
ax1.plot(thr_df["umbral"], thr_df["precision"], "s-", color=COLOR_OK, label=
ax1.plot(thr_df["umbral"], thr_df["pct_rechazados"], "^--", color="#7f8c8d",
ax1.axvline(CHOSEN_THRESHOLD, color="black", ls=":", lw=1.2, label=f"Elegidc
ax1.set_xlabel("Umbral")
ax1.set_ylabel("Tasa")
ax1.set_ylim(0, 1.05)
```

```
ax1.legend(loc="upper right")
ax1.set_title("Recall / Precision / % rechazados según umbral (HGB)")

ax2 = ax1.twinx()
ax2.plot(thr_df["umbral"], thr_df["costo_ilustrativo"], "D-", color="#8e44ac
ax2.set_ylabel("Costo ilustrativo (5×FN + 1×FP)")
ax2.legend(loc="center right")
fig.tight_layout()
fig.savefig(FIG / "15_umbrales_tradeoff.png", dpi=140, bbox_inches="tight")
plt.show()
```

```
/tmp/ipykernel_81677/2793125311.py:18: UserWarning: FigureCanvasAgg is non-i
nteractive, and thus cannot be shown
  plt.show()
```

In [16]:
```
# Predicción final con umbral elegido
y_pred_final = (y_proba_hgb >= CHOSEN_THRESHOLD).astype(int)
print(f"=== HGB con umbral {CHOSEN_THRESHOLD:.2f} ===")
print(classification_report(y_test, y_pred_final, target_names=["Sin mora",
print("Matriz de confusión:")
print(confusion_matrix(y_test, y_pred_final))

fig, ax = plt.subplots(figsize=(5.5, 4.5))
ConfusionMatrixDisplay(
    confusion_matrix(y_test, y_pred_final),
    display_labels=["Sin mora", "Mora"],
).plot(ax=ax, cmap="Oranges", colorbar=False)
ax.set_title(f"HGB — umbral {CHOSEN_THRESHOLD:.2f}")
fig.tight_layout()
fig.savefig(FIG / "16_confusion_umbral_elegido.png", dpi=140, bbox_inches="t
plt.show()
```

```
=== HGB con umbral 0.70 ===
              precision    recall  f1-score   support

    Sin mora       0.97      0.91      0.94     27995
        Mora       0.33      0.59      0.43      2005

    accuracy                           0.89     30000
   macro avg       0.65      0.75      0.68     30000
weighted avg       0.93      0.89      0.91     30000

Matriz de confusión:
[[25615  2380]
 [  813  1192]]
```

```
/tmp/ipykernel_81677/663554419.py:16: UserWarning: FigureCanvasAgg is non-in
teractive, and thus cannot be shown
  plt.show()
```

# 8. Importancia de variables (permutación) vs. ranking del EDA

**Permutation importance:** barajamos una feature y medimos cuánto cae el AUC.
Si cae mucho, la feature era importante. Es más fiel al modelo que el IV univariado,

pero deberían apuntar en la misma dirección.

In [17]:
```
# Permutation importance sobre el HGB (métrica AUC)
perm = permutation_importance(
    hgb,
    X_test,
    y_test,
    n_repeats=10,
    random_state=RANDOM_STATE,
    scoring="roc_auc",
    n_jobs=-1,
)

imp = pd.DataFrame({
    "feature": FEATURES,
    "importance_mean": perm.importances_mean,
    "importance_std": perm.importances_std,
}).sort_values("importance_mean", ascending=False)

# Traemos el ranking IV del EDA
eda_rank = pd.read_csv(REPORTS / "ranking_iv_auc.csv")
merged = imp.merge(
    eda_rank.rename(columns={"variable": "feature", "IV": "IV_eda", "AUC_uni
    on="feature",
    how="left",
)
print("=== Permutation importance (HGB, caída de AUC) ===")
print(merged.round(4).to_string(index=False))
merged.to_csv(REPORTS / "permutation_importance_hgb.csv", index=False)
```

```
=== Permutation importance (HGB, caída de AUC) ===
                feature  importance_mean  importance_std  IV_eda  AUC_eda  f
uerza_IV
          total_atrasos           0.1210          0.0026  1.3483   0.7680 mu
y fuerte
 uso_credito_revolvente           0.0723          0.0012  1.1134   0.7778 mu
y fuerte
                   edad           0.0106          0.0012  0.2592   0.6353
medio
lineas_credito_abiertas           0.0069          0.0005  0.0669   0.5445
débil
            ratio_deuda           0.0051          0.0007  0.0737   0.5238
débil
        ingreso_mensual           0.0020          0.0004  0.0670   0.5710
débil
 prestamos_hipotecarios           0.0020          0.0002  0.0121   0.5373
inútil
 codigo_atraso_especial           0.0011          0.0000  0.0391   0.5069
débil
           dependientes           0.0004          0.0002  0.0250   0.5476
débil
     uso_revolvente_gt1           0.0000          0.0000  0.2421   0.5542
medio
     ratio_deuda_dudoso           0.0000          0.0000  0.0077   0.5170
inútil
       ingreso_faltante          -0.0001          0.0001  0.0077   0.5170
inútil
```

In [18]:
```
fig, ax = plt.subplots(figsize=(9, 5.5))
plot_i = imp.sort_values("importance_mean")
ax.barh(
    plot_i["feature"],
    plot_i["importance_mean"],
    xerr=plot_i["importance_std"],
    color="#16a085",
    alpha=0.9,
)
ax.set_xlabel("Caída media de AUC al permutar")
ax.set_title("Importancia por permutación — HistGradientBoosting")
fig.tight_layout()
fig.savefig(FIG / "17_permutation_importance.png", dpi=140, bbox_inches="tig
plt.show()
```

```
/tmp/ipykernel_81677/3471861123.py:14: UserWarning: FigureCanvasAgg is non-i
nteractive, and thus cannot be shown
  plt.show()
```

# 9. Conclusiones y recomendaciones de negocio

In [19]:
```
# Números reales para el texto de conclusiones
m_log = metrics.loc[metrics["modelo"] == "Logística balanceada"].iloc[0]
m_hgb = metrics.loc[metrics["modelo"] == "HistGradientBoosting"].iloc[0]
m_base = metrics.loc[metrics["modelo"] == "Baseline (siempre 0)"].iloc[0]
best_row = thr_df.loc[thr_df["umbral"] == CHOSEN_THRESHOLD].iloc[0]
top_feats = list(imp.head(5)["feature"])
```

```
lines = []
lines.append("### Conclusiones (para un gerente de crédito / fintech)")
lines.append("")
lines.append(
    f"1. **La accuracy engaña:** el baseline «nadie cae en mora» logra accur
    f"pero recall {m_base['recall']:.0%}. No sirve para gestionar riesgo."
)
lines.append("")
lines.append(
    f"2. **Ambos modelos ordenan riesgo mucho mejor que el azar.** "
    f"AUC logística={m_log['auc']:.3f}; AUC HistGradientBoosting={m_hgb['auc
    f"El HGB gana por poco y captura no-linealidades."
)
lines.append("")
lines.append(
    f"3. **Con umbral 0.5 el recall del HGB es {m_hgb['recall']:.1%}** "
    f"(precision {m_hgb['precision']:.1%}). Bajando el umbral capturamos más
)
lines.append("")
lines.append(
    f"4. **Umbral recomendado (supuesto de costo 5×FN + 1×FP):** {CHOSEN_THR
    f"recall {best_row['recall']:.1%}, precision {best_row['precision']:.1%}
    f"rechaza/revisa al {best_row['pct_rechazados']:.1%} de la cartera de te
    f"Esto es un ejemplo: el área de riesgo debe fijar los pesos reales."
)
lines.append("")
lines.append(
    f"5. **Lo que más mueve el modelo:** {', '.join(top_feats)}. "
    "Coincide con el EDA: historial de atrasos y uso revolvente mandan; edad
)
lines.append("")
lines.append("6. **Recomendaciones prácticas:**")
lines.append("   - Usar el score HGB (o la logística si se prioriza interpre
lines.append("   - Regla dura: si `codigo_atraso_especial=1`, revisión manua
lines.append("   - Monitorear el % rechazado y el recall en producción; reca
lines.append("   - Próximo paso de portafolio: dashboard Looker Studio con s
print("\n".join(lines))
```

```
### Conclusiones (para un gerente de crédito / fintech)

1. **La accuracy engaña:** el baseline «nadie cae en mora» logra accuracy 9
3.3% pero recall 0%. No sirve para gestionar riesgo.

2. **Ambos modelos ordenan riesgo mucho mejor que el azar.** AUC logística=
0.859; AUC HistGradientBoosting=0.868. El HGB gana por poco y captura no-lin
ealidades.

3. **Con umbral 0.5 el recall del HGB es 78.6%** (precision 21.4%). Bajando
el umbral capturamos más defaulters a costa de más falsos positivos.

4. **Umbral recomendado (supuesto de costo 5×FN + 1×FP):** 0.70 → recall 59.
5%, precision 33.4%, rechaza/revisa al 11.9% de la cartera de test. Esto es
un ejemplo: el área de riesgo debe fijar los pesos reales.

5. **Lo que más mueve el modelo:** total_atrasos, uso_credito_revolvente, ed
ad, lineas_credito_abiertas, ratio_deuda. Coincide con el EDA: historial de
atrasos y uso revolvente mandan; edad e ingreso ayudan.

6. **Recomendaciones prácticas:**
   - Usar el score HGB (o la logística si se prioriza interpretabilidad) com
o **pre-screening**.
   - Regla dura: si `codigo_atraso_especial=1`, revisión manual / rechazo (m
ora histórica ~54%).
   - Monitorear el % rechazado y el recall en producción; recalibrar umbral
con costos reales.
   - Próximo paso de portafolio: dashboard Looker Studio con segmentos SQL +
distribución del score.
```

# Cierre

| Entregable | Estado |
|---|---|
| Baseline + logística + HGB | ✅ |
| ROC, PR, matrices, umbrales | ✅ |
| Permutation importance | ✅ |
| Recomendaciones de negocio | ✅ |

Siguiente capa del portafolio (fuera de este notebook): consultas SQL de segmentación y un dashboard en Looker Studio.